

COMP1531

Software Engineering Fundamentals

Week 2

Requirement Engineering

In This Lecture

— — —

Why?

- The most important part of building a system is figuring out what you need to do

What?

- Requirements
- Functional V Non-functional
- Requirements Engineering

Requirement Engineering In Software Development Lifecycle (SDLC)

— — —



Requirements

— — —

IEEE defines a requirement as:

- A condition or capability needed by a user to solve a problem or achieve an objective
- We would also describe requirements as:
 - Agreement of work to be completed by all stakeholders
 - Descriptions and constraints of a proposed system

Requirements

— — —

Why important?

- Foundation for Software Development
- Cost and Time
- Basis for Verification
- Helps Manage Scope



There has to be a way to authenticate.

I want to build a support system with live chat, contact form, help and case management.

I want to store user's Facebook ID in the database.

I want to view a chart that shows a percentage of time spent on particular websites.



Top 10 Worst Requirement

— — —

- #10 – “It has to be robust”
- #9 – “Easy to use”
- #8 – “X cannot change”
- #7 – “Make it accessible”
- #6 – “Reporting”
- #5 – “The system has to be bug-free”
- #4 – “The system shall work just like the previous one, but on a new platform”
- #3 – “Round-the-clock availability”
- #2 – “Response time should be less than X seconds”



And The Absolute Winner Is

— — —

#1 – “The system must have good usability”

Good Requirement Statements

The Online Banking System shall allow the Internet user to access her current account balance in less than 5 seconds.

1. Defines the system under discussion
2. Verb with a correct identifier (shall/may)
3. Positive end-result

Functional V Non-Functional

— — —

- Functional Requirements specify a specific capability/service that the system should (not) do. It's what the system does.
- Non-functional Requirements place a constraint on how the system can achieve that. Typically this is a performance characteristic. Often apply to the system as a whole rather than individual features or services.

[Some more reading on the topic](#)

Functional V Non-Functional

For example:

- Functional Requirement: The system shall send a notification to all users whenever there is a new post, or someone comments on an existing post
- Non-functional Requirement: The system shall send emails no later than 30 minutes after from such an activity

Functional V Non-Functional

— — —

One more example:

- A system that is designed for use in clinics attended by patients and records details of their consultations and conditions.

Functional requirements:

- A user shall be able to search the appointments lists for all clinics.
- The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.

Problems arise when requirements are not precisely stated. Consider the term ‘search’ in requirement 1.

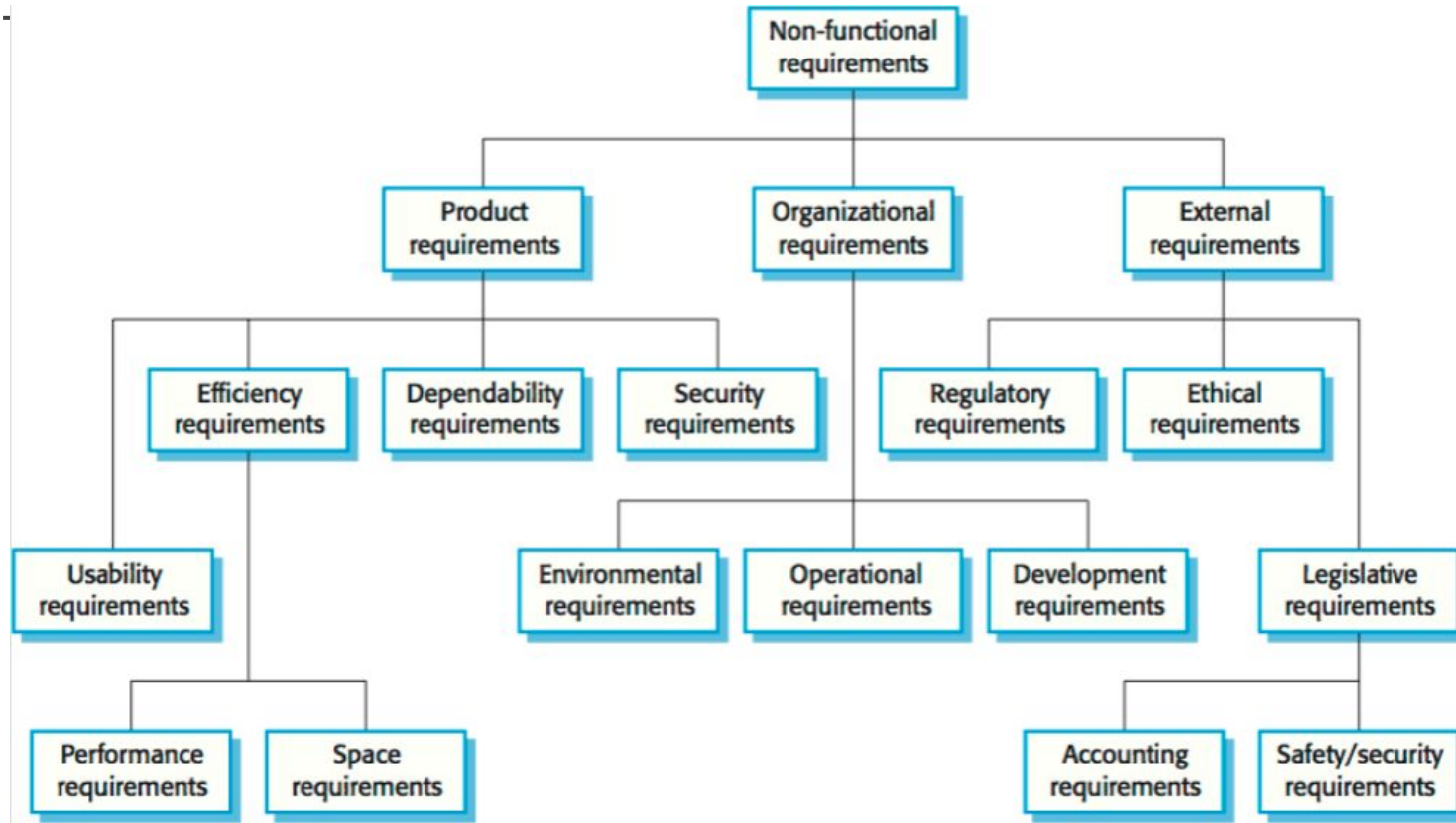
In principle, requirements should be both complete and consistent.

Functional V Non-Functional

Non-Functional requirements:

- *Product requirements:* The system shall be available to all clinics during normal working hours (Mon-Fri, 08.30-17.30). Downtime within normal working hours shall not exceed five seconds in any one day.
- *Organisational requirements:* Users of the system shall authenticate themselves using their health authority identity card.
- *External requirements:* The system shall implement patient privacy provisions as set out in HStan-03-2006-priv.

Functional V Non-Functional



Functional V Non-Functional

Key Points:

- Requirements for a software system set out what the system should do and define constraints on its operation and implementation.
 - Functional requirements are statements of the services that the system must provide or are descriptions of how some computations must be carried out.
 - Non-functional requirements often constrain the system being developed and the development process being used.

Quiz Time

Q1. Which of the following is a functional requirement for an online banking system?

- A. The system shall be available 99.9% of the time.
- B. The system shall allow users to transfer money between accounts.
- C. The system interface shall load within 2 seconds.
- D. The system shall support 10,000 concurrent users.

Quiz Time

Q2. Which of the following best describes a non-functional requirement?

- A. The system shall send a confirmation email after each purchase.
- B. The system shall support password reset.
- C. The website shall be accessible to visually impaired users.
- D. The system shall allow admin users to delete records.

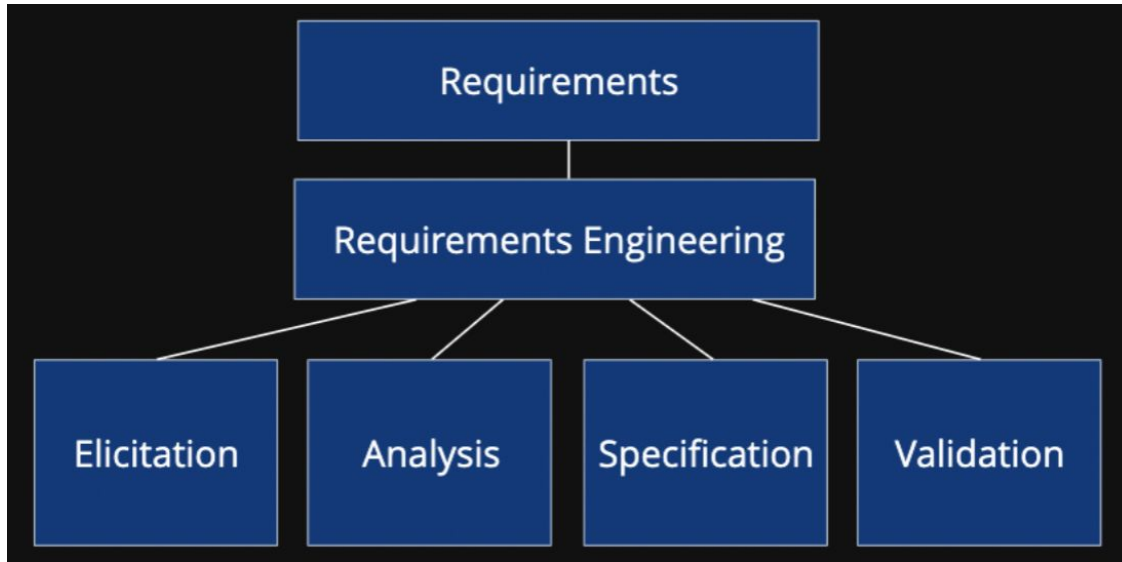
Quiz Time

Q3. Which statement is true about functional and non-functional requirements?

- A. Functional requirements describe how well a system performs a task.
- B. Non-functional requirements describe what the system should do.
- C. Functional requirements describe what the system should do;
Non-functional describe how well it performs it.
- D. Both types describe system behavior equally.

Requirements Engineering

Requirements don't just appear out of thin air. We have to derive them, and to do that we apply the process of requirements engineering.



Requirements Engineering

Requirements Engineering is:

- A set of activities focused on identifying the purpose and goal of a software system
- A negotiation process where stakeholders agree on what they want. Stakeholders include:
 - May involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc.

Requirements Engineering

Requirements engineering often follows a logical process across 4 steps:

1. Elicitation of raw requirements from stakeholders
2. Analysis of requirements
3. Formal specification of requirements
4. Validation of requirements

In practice, RE is an iterative activity in which these processes are interleaved.

Step 1: Elicitation

Questions and discovery

- Market Research
- Interviews with Stakeholders
- Focus groups
- Asking questions "What if? What is?"

Software engineers work with a range of system stakeholders to find out about the application domain, the services that the system should provide, the required system performance, hardware constraints, other systems, etc.

Step 2: Analysis

Building the picture

- Identify dependencies, conflicts, risks
- Establish relative priorities
- Usually done through:
 - User stories
 - Use cases
 - Conceptual modeling

Step 3: Specification

Refining the picture

- Establishing the right sense of granularity
 - There is no perfect way to granulate
- Often the stage of breaking up into functional and non-functional.
- It is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it.
- The software requirements document is an agreed statement of the system requirements. It should be organized so that both system customers and software developers can use it.

Step 4: Validation

Checking you haven't gotten lost

Going back to stakeholders and ensuring requirements are correct.

Challenges During RE

— — —

What are some challenges we may face while engaging in Requirements engineering?

- Requirements sometimes only understood **after** design/build has begun
- Clients/customers sometimes **don't know what they want**
- Clients/customers sometimes **change** their mind
- Developers might not understand the subject domain
- **Limited access** to stake holders
- Jumping into details or solutions too early (XY problem)

Quiz Time

Q1. Which activity is not part of requirements elicitation?

- A. Conducting interviews with stakeholders
- B. Observing users perform current tasks
- C. Writing unit tests for requirements
- D. Facilitating workshops for requirement discovery

Quiz Time

Q2. Conflicts between requirements are best handled by:

- A. Ignoring them
- B. Using negotiation and prioritization
- C. Assigning them to different developers
- D. Rewriting them as non-functional requirements

Quiz Time

Q3. A good requirement is one that is:

- A. Ambiguous and general
- B. Specific, measurable, and testable
- C. Based on assumptions
- D. Described using natural language only

Quiz Time

Q4. The goal of requirements validation is to:

- A. Ensure the system meets stakeholder needs
- B. Check for syntax errors in the Software Requirements Specification (SRS)
- C. Generate automated test cases
- D. Optimize code performance

Feedback

— — —



Or go to the [form here](#)

COMP1531

Software Engineering Fundamentals

Week 2

Requirements - Use Cases, User Stories

In This Lecture

— — —

Why?

- Requirements can be very human, and express complex flows. We need ways to model this.

What?

- User Stories
- Use Cases

Use Cases - Why?

— — —

- The traditional approach for **specifying functionality** is to specify each function that the system should provide.
- Use cases specify the functionality of a system by specifying the **behaviour of the system**, captured as interactions of the users with the system.
- UCs not only document requirements, as their form is like **storytelling** and uses text, both of which are easy and natural with different stakeholders, they also are a good medium for discussion and brainstorming.

Use Cases

— — —

- Represent **a dialogue** between the user and the system, with the aim of helping the user achieve a business goal.
- The user initiates actions and the system responds with reactions.
- They consider systems as a **black box**, and are only focused on high level understanding of flow.

Why Use Cases Are Powerful

— — —

Human-friendly → Easy to read and understand

Story-driven → Feels natural for all stakeholders

Collaborative → Great for discussion & brainstorming

Clear context → Shows *who does what* and *why*

Use Cases

— — —

Generally you can represent use cases as:

- Informal list of steps (written)
- Diagrammatic (visual)
- There is a range of different approaches that can be taken too, e.g. Cockburn style (not required reading)

Use Cases

— — —

HOW?

Software system requirements are usually written in the form of statements
“The system shall ...” or “The system should ...”

Use Cases

Example 1:

Identifier	Priority	Requirement
REQ1	5	The system shall keep the door locked at all times, unless commanded otherwise by authorized user. When the lock is disarmed, a countdown shall be initiated at the end of which the lock shall be automatically armed (if still disarmed).
REQ2	2	The system shall lock the door when commanded by pressing a dedicated button.
REQ3	5	The system shall, given a valid key code, unlock the door and activate other devices.
REQ4	4	The system should allow mistakes while entering the key code. However, to resist “dictionary attacks,” the number of allowed failed attempts shall be small, say three, after which the system will block and the alarm bell shall be sounded.
REQ5	2	The system shall maintain a history log of all attempted accesses for later review.
REQ6	2	The system should allow adding new authorized persons at runtime or removing existing ones.
REQ7	2	The system shall allow configuring the preferences for device activation when the user provides a valid key code, as well as when a burglary attempt is detected.
REQ8	1	The system should allow searching the history log by specifying one or more of these parameters: the time frame, the actor role, the door location, or the event type (unlock, lock, power failure, etc.). This function shall be available over the Web by pointing a browser to a specified URL.
REQ9	1	The system should allow filing inquiries about “suspicious” accesses. This function shall be available over the Web.

Marsic, I. (2012). Software Engineering. Rutgers University. Retrieved from https://eceweb1.rutgers.edu/~marsic/books/SE/book-SE_marsic.pdf

Use Cases

— — —

HOW?

- Actors and goals.
- Main success scenarios.
- Failure conditions.
- Failure handling.

Use Cases

Example 2a:

- *UC1: Put an item for auction*

Primary Actor: Seller

Precondition: Seller has logged in

Main Success Scenario:

1. Seller posts an item (its category, description, picture, etc.) for auction
2. System shows past prices of similar items to seller
3. Seller specifies the starting bid price and a date when auction will close
4. System accepts the item and posts it

Exception Scenarios:

- 2 a) There are no past items of this category
 - System tells the seller this situation

Use Cases

Example 2b:

– *UC2: Make a bid*

Primary Actor: Buyer

Precondition: The buyer has logged in

Main Success Scenario:

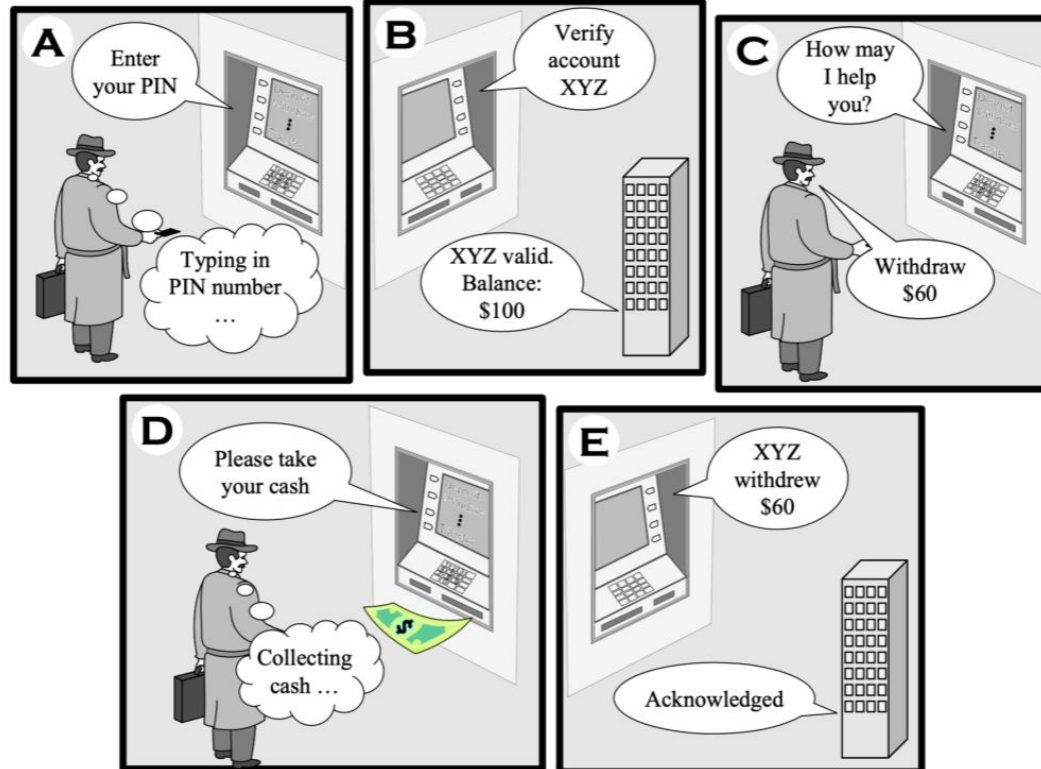
1. Buyer searches or browses and selects some item
2. System shows the rating of the seller, the starting bid, the current bids, and the highest bid; asks buyer to make a bid
3. Buyer specifies a bid price
4. System accepts the bid; Blocks funds in bidders account
5. System updates the max bid price, informs other users, and updates the records for the item

Exception Scenarios:

- 3 a) The bid price is lower than the current highest
 - System informs the bidder and asks to rebid
- 4 a) The bidder does not have enough funds in his account
 - System cancels the bid, asks the user to get more funds

Use-Case Diagrams

Example 3:



Use-Case List

— — —
We can provide a use case in written form.

Step 1. ATM asks customer for pin

Step 2. Customer enters pin

Step 3. ATM asks bank to verify pin and account

Step 4. Bank informs ATM of validity and balance of account

Step 5. ATM asks customer what action they wish to take

Step 6. Customer asks to withdraw an amount of money

Step 7. ATM Dispenses money to customer

Step 8. ATM informs bank of withdrawal

User Stories

— — —

Overview

- User Stories are a method of requirements engineering used to inform the development process and what features to build with the **user at the centre**.
- They are commonly used in Agile models.

User Stories

— — —

Structure

When a customer tells you what they want, try and express it in the form
As a <**type of user**>, I want <**some goal**> so that <**some reason**>

Example, They say:

- A student can purchase monthly parking passes online

But your story becomes:

- As a **student**, I want to **purchase a parking pass** so that I can drive to school

User Stories

— — —

A user story is intended to contain just enough information so that the developers can produce a reasonable estimate of the effort to implement it.

The aim is to avoid some of the waste that often happens in projects where detailed requirements are gathered early but become invalid before the work begins.

User Stories

— — —

Attributes

- Are written in non-technical language
- Are focused on keeping the customer at the core of the experience
- Are user-goal focused, not product-feature focused
 - User stories ideally describe problems, not solutions

User Stories

— — —

Good properties of user stories:

- Independent: Can be developed/delivered independently
- Negotiable: avoid too much detail.
- Valuable: must hold some value to the client
- Estimable: we'll get to this in a later lecture
- Small: user story should be small
- Testable

User Stories

— — —

Activity

Let's write out some user stories for a to-do list!

User Stories

— — —

Activity

Let's write out some user stories for a to-do list!

- As a user, I want to **add a new task**, so that I can keep track of things I need to do.
- As a user, I want to **delete a task**, so that I can remove things I no longer need.
- As a user, I want to **mark a task as complete**, so that I can track my progress.
- As a user, I want to **set due dates**, so that I can manage deadlines.
- As a user, I want to **prioritize tasks**, so that I can focus on what's most important.
- As a user, I want to **set reminders**, so that I don't forget important tasks.
- As a user, I want to **search for tasks**, so that I can quickly find something.
- As a user, I want to **sync my tasks across devices**, so that I can access them anywhere.

User Stories

— — —

Further Reading

[Atlassian User Stories](#)



User Acceptance Criteria

— — —

Before a user story is implemented, an appropriate **acceptance procedure** must be written by the customer to determine whether the **goals of the user story have been fulfilled**.

We need a method to help us "test" whether a user story has been satisfied.

- Break down a user story into criteria that must be met for the user, or customer, to accept.
- Written in natural language.
- Can be refined before implementation



User Acceptance Criteria

For example: As a user, I want to use a search field to type a city, name, or street, so that I can find matching hotel options.

User Acceptance Criteria:

- The search field is placed on the top bar
- Search starts once the user clicks "Search"
- The field contains a placeholder with a grey-colored text: "Where are you going?"
- The placeholder disappears once the user starts typing
- Search is performed if a user types in a city, hotel name, street, or all combined
- The user can't type more than 200 symbol



User Acceptance Criteria

For example: As a user, I can log in through a social media account, because I always forget my passwords

User Acceptance Criteria:

- Can log in through Facebook
- Can log in through LinkedIn
- Can log in through Twitter



User Acceptance Criteria

— — —

- Acceptance criteria should **not be too broad** (but nor should they be too narrow).
- **Minimise technical detail.** They can be more technical than the story itself, but client still needs to understand them.
- While they can be updated during development, they should first be **written before** it starts.
- Acceptance Tests are tests that are performed to ensure acceptance criteria have been met.
- Not all acceptance criteria can easily be mapped to automated acceptance tests.
- Sometimes you need to setup User Acceptance Testing environments for real users to manually use it and provide feedback.
- Acceptance tests are **black-box tests**.

Alternative: Scenario Oriented

— — —

The Acceptance criteria from before are often referred to a rule-based AC. Sometimes it is preferable to have AC that describe a scenario

This can be done in the **Given/When/Then** format:

- Given some precondition
- When I do some action
- Then I expect some result

Alternative: Scenario Oriented

— — —

Example: Google Basic Search Functionality

As a user, I want to search for information, so that I can find relevant results quickly.

Acceptance Criteria:

- **Given** I am on the search page
When I enter a valid query and press Enter or click Search
Then I should see a list of relevant results within 1 second
- **Given** I submit an empty query
When I press Search
Then I should remain on the page and see a prompt or no results

Alternative: Scenario Oriented

— — —

Example: As a user, I want to be able to recover the password to my account, so that I will be able to access my account in case I forgot the password.

User Acceptance Criteria:

Scenario: Forgot password

- **Given:** The user has navigated to the login page
 - **When:** The user selected forgot password option
 - **And:** Entered a valid email to receive a link for password recovery
 - **Then:** The system sent the link to the entered email
-
- **Given:** The user received the link via the email
 - **When:** The user navigated through the link received in the email
 - **Then:** The system enables the user to set a new password

Which One To Use?

— — —

Rule-based acceptance criteria are simpler and generally work for all sorts of stories

Scenario-based AC work for stories that imply specific user actions, but don't work for higher-level system properties

Scenario-based AC are more likely to be implementable as test.

Further Reading

— — —

If you wish to know more about use cases, see here:

- <http://www.cs.otago.ac.nz/coursework/cosc461/uctempla.htm>
- <https://www.mountangoatsoftware.com/blog/the-two-ways-to-add-detail-to-userstories>
- <https://www.altexsoft.com/blog/business/acceptance-criteria-purposes-formats-andbest-practices/>
- <https://dzone.com/articles/acceptance-criteria-in-software-explanation-example>

Feedback

— — —



Or go to the [form here](#)

COMP1531

Software Engineering Fundamentals

Week 2

Requirements - Validation

In This Lecture

— — —

Why?

- It's important to understand the difference between verification and validation

What?

- Validation
- User Acceptance Testing

Let's Compare

*ISO/IEC/IEEE 29148:2018

Verification

- Verification in a system life cycle context is a set of activities that compares a product of the system life cycle **against the required characteristics for that product**. This may include, but is not limited to, specified requirements, design description and the system itself.

Validation

- Validation in a system life cycle context is a set of activities ensuring and gaining confidence that a system is **able to accomplish its intended use, goals and objectives**.

Let's Compare

— — —

Verification

- System will be built right

Validation

- Right system will be built

Why Do We Validate?

— — —

- The aim is to pick up any problems before resources are committed to addressing the requirements.
- Requirements validation is concerned with the process of examining the requirements document to ensure that it defines the right software (that is, the software that the users expect).

How Do We Validate?

— — —

- Requirements Reviews
- Prototyping
- Acceptance Tests

User Acceptance Testing

— — —

- [ISTQB](#) defines acceptance testing as **formal testing with respect to user needs, requirements, and business processes** conducted to determine whether or not a system satisfies the acceptance criteria and to enable the user, customers or other authorized entity to determine whether or not to accept the system.
- This testing is always black-box, and is typically tested on the customer or user themselves.
- Can either be performance based (trying to reach specific outcome) or stress testing. Sometimes you'll hear this testing environment be referred to as UAT.

Feedback

— — —



Or go to the [form here](#)